

Evaluation of the Existing LLM / RAG Approach

Assessment of the current DiscountMate Recipe RAG chatbot and options for extending it with structured outputs, tool calling and agent orchestration.

Prepared by	Sunain Mushtaq
Project	DiscountMate - Deep Learning / AI Stream
Task	DL-06-T6
Focus	Existing chatbot evaluation and migration direction

Executive conclusion

The current Recipe RAG system is a useful foundation and should not be discarded. Its semantic retrieval, recipe index, BM25 matching, database grounding and existing chatbot integration are reusable. However, the current design is tightly coupled to recipe retrieval and does not support dynamic tool selection for broader shopping actions such as price comparison or substitution. The recommended direction is to convert the chatbot into an LLM-agent architecture with structured tool calls, while retaining the existing Recipe RAG as a specialised recipe tool.

Repository areas reviewed

- Backend/ml-service/recipe_rag/rag_pipeline.py
- Backend/ml-service/app.py
- Backend/src/controllers/ml.controller.js
- Backend/src/routers/ml.router.js
- Frontend/app/(tabs)/RecipeBot.tsx

1. Current Chatbot Architecture

The current DiscountMate chatbot is a specialised **Recipe Retrieval-Augmented Generation (RAG)** system. It is not a general shopping assistant. The workflow is designed around retrieving relevant recipes, passing those recipes to an instruction-tuned LLM, and then grounding suggested grocery products using live product data.

```
User Query
↓
React Native RecipeBot UI
↓
Express API Proxy
↓
Flask ML Service
↓
all-mpnet-base-v2 semantic retrieval
↓
Recipe vector index
↓
OpenRouter LLM cascade → Hugging Face fallback
↓
Generated recipe answer
↓
BM25 + MongoDB product grounding
↓
Product cards in the frontend
```

1.1 Models and techniques currently used

Component	Current approach	Purpose
Embedding model	all-mpnet-base-v2	Converts user queries and recipe text into embeddings for semantic retrieval.
Primary LLM provider	OpenRouter	Generates natural-language responses using a cascade of instruction-tuned models.
Fallback LLM provider	Hugging Face	Provides additional models if the OpenRouter models are unavailable or fail.
Lexical ranking	BM25	Improves exact product/ingredient matching where lexical information matters.
Product data	MongoDB	Provides real product names, prices/images and grounding to reduce hallucination.
Conversation state	In-memory session history	Supports short multi-turn recipe conversations.

1.2 Current LLM model cascade

The code currently defines several OpenRouter models and several Hugging Face fallback models. This provides redundancy, but it also means that different requests may be answered by different model families with different response behaviour, latency and structured-output reliability.

OpenRouter:

- nvidia/nemotron-nano-12b-v2-v1:free
- nvidia/nemotron-3-super-120b-a12b:free
- nvidia/nemotron-3-nano-30b-a3b:free

Hugging Face fallback:

- MiniMaxAI/MiniMax-M2.7
- openai/gpt-oss-120b
- Qwen/Qwen2.5-7B-Instruct
- meta-llama/Llama-3.1-8B-Instruct

2. Strengths of the Existing Approach

2.1 Strong semantic retrieval

The use of `all-mpnet-base-v2` is appropriate for recipe retrieval because it can match queries based on meaning rather than exact words. A request such as “*something quick with chicken*” can still retrieve suitable recipes even when the wording differs from the indexed recipe title or description.

Recommendation: keep the current embedding model initially. There is no strong reason to replace a working semantic retrieval model during the first stage of the chatbot refactor.

2.2 RAG reduces unsupported generation

The chatbot does not ask the LLM to invent recipes from general model knowledge. Instead, relevant recipe records are retrieved first and passed into the generation stage. This is a suitable pattern for DiscountMate because answers should be based on application data rather than general internet knowledge.

2.3 Product grounding is a strong design principle

The existing implementation uses live MongoDB product documents and BM25 matching to ground suggested grocery products. Product names and product data do not need to come directly from the LLM. This is especially important for future price comparison because the LLM must never become the source of truth for prices.

Principle to retain: The LLM interprets the user request; the DiscountMate data layer provides the factual answer.

2.4 Hybrid lexical + semantic matching is useful

Product search differs from recipe search. Exact brand names, pack sizes, variants and product terms are often critical. BM25 is therefore useful alongside semantic embeddings. A hybrid design can preserve exact product terms while still handling natural-language variation.

2.5 Existing integration reduces redevelopment effort

The project already contains a working frontend chatbot component, an Express proxy, Flask endpoints, session handling, LLM integration, RAG retrieval and product-card rendering. Reusing these parts reduces unnecessary redevelopment and allows the team to focus on the new agent and shopping capabilities.

3. Limitations and Risks

3.1 Architecture is recipe-centric

The most important limitation is that the entire workflow assumes that the user wants a recipe. The system prompt explicitly defines the assistant as a recipe assistant and instructs it to reject unrelated questions. This makes the architecture unsuitable for broader DiscountMate actions such as product search, price comparison and product substitution.

Current pattern: query → retrieve recipes → call LLM → answer

Required pattern: query → understand intent → select tool → retrieve application data → validate → answer

3.2 No true dynamic tool calling

The current sequence is mostly predetermined. The LLM does not dynamically choose functions such as `compare_prices()`, `search_products()` or `find_substitutes()`. This is the main functional gap between the old Recipe RAG and the proposed DiscountMate shopping assistant.

3.3 Large LLM fallback chain may produce inconsistent behaviour

Using many different model families improves availability, but it reduces consistency. Different models may vary in JSON reliability, function/tool selection, latency, refusal behaviour and formatting. This becomes more important once the assistant must emit validated structured tool arguments.

Recommendation: move toward a small, tested model set - ideally one primary tool-calling model and one fallback - instead of relying on a long free-model cascade.

3.4 Too much logic is coupled inside the RAG pipeline

The current RAG pipeline combines retrieval, prompting, external LLM calls, product matching, MongoDB access, BM25 ranking and session state. This creates a large component with several responsibilities, making testing and future database migration harder.

3.5 Direct MongoDB dependency

The product resolver connects directly to MongoDB. This means chatbot logic is aware of storage implementation details. If DiscountMate moves toward PostgreSQL or changes backend schemas, chatbot code must also be changed. The new architecture should access product data through stable tools or backend service contracts.

3.6 External API and free-model reliability

The current generation stage depends on external model APIs. Free or shared models may have variable latency, rate limits, temporary unavailability or provider-specific behaviour. These risks are acceptable for development, but the new design should explicitly measure latency and structured-output reliability.

3.7 Conversation state is not production-grade

Conversation history is stored in memory and limited to a small number of turns. This is acceptable for a prototype but is not ideal for a horizontally scaled deployment where different requests may reach different service instances.

4. Evaluation of the Current Retrieval Models

4.1 all-mpnet-base-v2

For recipe retrieval, all-mpnet-base-v2 is a sensible model to retain. It provides strong general semantic similarity, runs locally through SentenceTransformers, avoids embedding API cost and already has compatible indexes in the current implementation.

Criterion	Assessment
Semantic quality	Strong for natural-language recipe queries.
Cost	Runs locally; no per-query embedding API cost.
Integration effort	Already integrated and indexed.
Latency	Reasonable for local embedding inference, depending on host resources.
Suitability for product matching	Useful, but should not be used alone because product names depend heavily on exact brand, size and variant tokens.
Decision	Keep for recipe retrieval; evaluate as part of hybrid product matching.

4.2 Product matching should remain hybrid

For product search and price comparison, exact lexical information is often more important than broad semantic similarity. For example, *Coke Zero 2L*, *Coke Zero 375mL* and *Coke Original 2L* are semantically close but should not automatically be treated as equivalent products.

Recommended product matching pipeline:

1. Normalise text and extract product entities
2. Apply exact/lexical matching for brand, pack size and variant
3. Use BM25 for strong keyword ranking
4. Use embedding similarity where natural-language wording differs
5. Apply structured filters before final comparison

This approach is better suited to price comparison than relying on embeddings alone.

5. Evaluation of the Current LLM Strategy

The existing LLM cascade is acceptable for experimentation and general answer generation, but it should be reconsidered for the new agent system. Tool calling requires more predictable behaviour than free-form text generation.

Requirement	Importance for new chatbot	Current concern
Structured JSON output	High	Different fallback models may format outputs differently.
Tool/function calling	High	Current flow does not depend on native tool selection.
Instruction following	High	Quality varies across models.
Latency	High	Cascade/fallback attempts can increase response time.
Availability	High	Cascade is strong here because several models are available.
Cost	Medium	Free models reduce cost but may be less reliable.

5.1 Recommended direction for model evaluation

Rather than selecting an LLM only by general response quality, the team should compare candidate models specifically on the tasks required by the new assistant:

- Correct tool selection for product search, price comparison, recipe retrieval and substitution.
- Correct extraction of product name, brand, retailer, pack size and other arguments.
- Valid JSON / schema compliance.
- Ability to request clarification when required information is missing.
- Latency under normal development conditions.
- Availability, cost and external API limits.

A smaller number of tested models is preferable to a large fallback chain for production-like agent behaviour.

5.2 Structured outputs should be added

The LLM should not directly generate factual shopping answers before retrieving DiscountMate data. Instead, it should first convert the natural-language request into a structured action.

```
{ "action": "price_comparison", "arguments": { "product_name": "Coke Zero", "pack_size": "2L" } }
```

The selected tool then retrieves current application data. Only after receiving that result should the assistant produce the user-facing natural-language answer. This reduces hallucination and makes tool behaviour easier to test.

6. Recommended Agent Architecture

The current RAG system should be refactored rather than replaced. The key change is to place an LLM agent and orchestration layer above a set of specialised tools.

```
User
↓
Existing / Updated Chat UI
↓
Backend API
↓
LLM Agent
↓
LangGraph Orchestration
↓
Tool Router
■■■ search_products()
■■■ compare_prices()
■■■ get_product_details()
■■■ find_substitutes()
■■■ search_recipes()
↓
Existing Recipe RAG

Tools → MCP / Backend API → DiscountMate data services → Database
```

6.1 Role of the existing Recipe RAG

The Recipe RAG should remain in the system but become a specialised tool. This preserves the current embedding model, recipe index and recipe generation behaviour while allowing non-recipe shopping requests to bypass recipe retrieval entirely.

Recommended conversion: Existing Recipe RAG → `search_recipes()` / `get_recipe()` tool.

6.2 Why LangGraph is useful

LangGraph is not necessary for the old recipe-only chatbot because the workflow is largely linear. It becomes more useful once the assistant supports multiple tools, clarification, application context and multi-step requests.

```
Understand request
↓
Is information missing?
■■■ Yes → ask clarification → re-enter workflow
■■■ No → select tool
↓
execute tool
↓
validate result
↓
another tool required?
■■■ Yes → execute next tool
■■■ No → generate final response
```

This makes control flow explicit and testable instead of relying entirely on a single LLM prompt.

6.3 MCP / backend boundary

The chatbot should not need to know whether product data comes from MongoDB, PostgreSQL or another storage system. Tool contracts should isolate the agent from database implementation. MCP or a stable backend API can provide that boundary.

Recommended separation: LLM decides what it needs → tool requests structured data → backend/database provides factual result.

7. Migration Strategy

The safest approach is an incremental refactor rather than a full rewrite.

Stage 1 - Audit and preserve

- Keep the existing RecipeBot frontend and API path during early development.
- Keep the recipe index and all-mpnet-base-v2 retrieval.
- Keep BM25 and current product matching logic as reference implementations.
- Document the current MongoDB dependencies and RAG interfaces.

Stage 2 - Add structured agent layer

- Introduce a general DiscountMate system prompt.
- Define actions and JSON schemas.
- Add structured tool calling and validation.
- Implement price comparison as the first new end-to-end action.

Stage 3 - Modularise existing capabilities

- Wrap Recipe RAG as a recipe tool.
- Move product access behind a tool/backend interface.
- Separate retrieval, tool logic and session/orchestration responsibilities.

Stage 4 - Add orchestration and context

- Introduce LangGraph for routing, clarification and multi-step workflows.
- Add current-screen/current-product application context.
- Add substitution and future tools.

Stage 5 - Evaluate and harden

- Measure tool-selection accuracy and argument extraction accuracy.
- Measure retrieval accuracy and product match quality.
- Measure latency and provider failure rates.
- Add regression tests to ensure recipe functionality still works.

8. Final Component Decisions

Component	Decision	Reason
all-mpnet-base-v2	KEEP	Already integrated and suitable for semantic recipe retrieval.
Recipe embedding/index files	KEEP	Useful existing asset; no need to rebuild immediately.
Recipe RAG	KEEP AS TOOL	Retain recipe capability while removing recipe-centric control over the whole chatbot.
BM25	KEEP / IMPROVE	Useful for exact product names, brands, variants and pack sizes.
Direct MongoDB access	REPLACE GRADUALLY	Move behind a backend/MCP contract to support PostgreSQL migration and cleaner separation.
Existing RecipeBot UI	REUSE	Already provides working chat UX and product-card rendering.
Large LLM cascade	SIMPLIFY / EVALUATE	Too many model families can make structured outputs and tool selection inconsistent.
Structured JSON outputs	ADD	Required for reliable, testable action/tool invocation.
Tool calling	ADD	Needed for product search, price comparison, substitution and recipe routing.
LangGraph	ADD FOR ORCHESTRATION	Useful once workflows include clarification, context and multiple tools.
MCP / backend abstraction	ADD	Separates the AI layer from storage and backend implementation details.
Recipe-only system prompt	REPLACE	Prevents the chatbot from acting as a general DiscountMate shopping assistant.

9. DL-06-T6 Conclusion

The existing DiscountMate Recipe RAG provides a useful technical foundation and should not be replaced entirely. Its semantic retrieval using `all-mpnet-base-v2`, database-grounded product matching, BM25 ranking and existing LLM integration can be reused. However, the current architecture is tightly coupled to recipe retrieval, relies on a large and potentially inconsistent model fallback chain, and does not support dynamic tool selection.

The recommended approach is to refactor the chatbot into an LLM-agent architecture where structured tool calling is used for actions such as product search, price comparison and product substitution. The existing Recipe RAG should be retained as a specialised recipe-search tool. LangGraph can then manage tool selection, clarification, multi-step workflows and conversation state, while MCP or backend APIs provide a consistent interface between chatbot tools and DiscountMate data.

This conclusion directly informs DL-06-T7, where the revised LLM-agent architecture, tool boundaries, LangGraph workflow and JSON contracts can be formally designed.

Prepared by: **Sunain Mushtaq**